

Experiment-1

(a) Given a stack of size N, implement PUSH and POP operations following the LIFO (Last In First Out) principle. Handle overflow and underflow conditions, and display the final contents of the stack.

Source Code:

```
#include <iostream>
using namespace std;
#define N 5
class Stack {
    int stack[N];
    int top;
public:
    Stack() {
        top = -1;
    }
    void push(int x) {
        if (top == N - 1) {
            cout << "Stack Overflow" << endl;
            return;
        }
        stack[++top] = x;
    }
    void pop() {
        if (top == -1) {
            cout << "Stack Underflow" << endl;
            return;
        }
        cout << "Popped element: " << stack[top--] << endl;
    }
    void display() {
        if (top == -1) {
            cout << "Stack is empty" << endl;
            return;
        }
        cout << "Final Stack: ";
        for (int i = top; i >= 0; i--)
            cout << stack[i] << " ";
        cout << endl;
    }
};
```

```

    }
};

int main() {
    Stack s;
    s.push(10);
    s.push(20);
    s.push(30);
    s.push(40);
    s.push(50);
    s.push(60);
    s.pop();
    s.pop();
    s.display();
    return 0;
}

```

Output:

```

PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> cd "c:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects\" ; if
($?) { g++ lab.cpp -o lab } ; if ($?) { .\lab }
Stack Overflow
Popped element: 50
Popped element: 40
Final Stack: 30 20 10
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects>

```

(b) Given an infix expression, convert it into its equivalent postfix expression using a stack by following operator precedence and associativity rules.

Source Code:

```

#include <iostream>
#include <stack>
#include <string>
#include <cctype>
using namespace std;
int precedence(char op) {
    if (op == '^')
        return 3;
    if (op == '*' || op == '/')
        return 2;
    if (op == '+' || op == '-')
        return 1;
    return 0;
}

```

```

}
bool isOperator(char c) {
    return c == '+' || c == '-' || c == '*' || c
== '/' || c == '^';
}
string infixToPostfix(string infix) {
    stack<char> s;
    string postfix;
    for (char c : infix) {
        if (isalnum(c)) {
            postfix += c;
        }
        else if (c == '(') {
            s.push(c);
        }
        else if (c == ')') {
            while (!s.empty() && s.top() != '(') { postfix
+= s.top();
            s.pop();
            }
            s.pop();
        }
        else if (isOperator(c)) {
            while (!s.empty() && s.top() != '(' &&
precedence(s.top()) >= precedence(c)) { postfix +=
s.top();
            s.pop();
            }
            s.push(c);
        }
        while (!s.empty()) {
            postfix += s.top();
            s.pop();
        }
        return postfix;
    }
}
int main() {
    string infix;
    cout << "Enter infix expression: ";

```

```

cin >> infix;
cout << "Postfix expression: "
<< infixToPostfix(infix) << endl;
return 0;
}

```

Output:



```

> cd "c:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects\" ; if
($?) { g++ lab.cpp -o lab } ; if ($?) { .\lab }
Enter infix expression: A/B*C^D
Postfix expression: AB/CD^*
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects>

```

(c) Evaluate the generated postfix expression using a stack and display the computed result.

Source Code:

```

#include <iostream>
#include <stack>
#include <string>
#include <cmath>
#include <cctype>
using namespace std;
int evaluatePostfix(string postfix) {
    stack<int> s;
    for (char c : postfix) {
        if (isdigit(c)) {
            s.push(c - '0');
        }
        else {
            int b = s.top();
            s.pop();
            int a = s.top();
            s.pop();
            switch (c) {
                case '+':
                    s.push(a + b);
                    break;
                case '-':
                    s.push(a - b);
                    break;
                case '*':
                    s.push(a * b);

```

```

break;
case '/':
s.push(a / b);
break;
case '^':
s.push(pow(a, b));
break;
}
}
}
return s.top();
}
int main() {
string postfix;
cout << "Enter postfix expression: ";
cin >> postfix;
cout << "Result: "
<< evaluatePostfix(postfix) << endl;
return 0;
}

```

Output:

```

PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> ./lab.exe
Enter postfix expression: AB*+C-
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> ./lab.exe
Enter postfix expression: 3 3 * + 4 -
Result: 3
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> █

```

Experiment-2

(a) Given a queue of size N, implement ENQUEUE and DEQUEUE operations using an array or linked list while following the FIFO (First In First Out) principle. Handle overflow and underflow conditions, and display the final state of the queue.

Source Code:

```
#include <iostream>
using namespace std;
#define N 5
class Queue {
    int q[N];
    int front, rear;
public:
    Queue() {
        front = -1;
        rear = -1;
    }
    void enqueue(int x) {
        if (rear == N - 1) {
            cout << "Queue Overflow" << endl;
            return;
        }
        if (front == -1)
            front = 0;
        q[++rear] = x;
    }
    void dequeue() {
        if (front == -1) {
            cout << "Queue Underflow" << endl;
            return;
        }
        cout << "Deleted element: " << q[front] << endl;    if
(front == rear)
            front = rear = -1;
        else
            front++;
    }
}
```

```

void display() {
    if (front == -1) {
        cout << "Queue is empty" << endl;
        return;
    }
    cout << "Final Queue: ";
    for (int i = front; i <= rear; i++)
        cout << q[i] << " ";
    cout << endl;
}
};

int main() {
    Queue q;
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);
    q.enqueue(60);
    q.dequeue();
    q.dequeue();
    q.display();
    return 0;
}

```

Output:

```

PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> g++ lab.cpp -o lab
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> ./lab.exe
Queue Overflow
Deleted element: 10
Deleted element: 20
Final Queue: 30 40 50
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects>

```

(b) Implement a circular queue using an array, where vacant positions created by deletions are reused by maintaining the circular property. Perform the given sequence of operations and display the final state of the circular queue.

Source Code:

```

#include <iostream>
using namespace std;
#define N 5

```

```

class CircularQueue {
    int q[N];
    int front, rear;
public:
    CircularQueue() {
        front = -1;
        rear = -1;
    }
    void enqueue(int x) {
        if ((rear + 1) % N == front) {
            cout << "Circular Queue Overflow" << endl;    return;
        }
        if (front == -1)
            front = rear = 0;
        else
            rear = (rear + 1) % N;
        q[rear] = x;
    }
    void dequeue() {
        if (front == -1) {
            cout << "Circular Queue Underflow" << endl;    return;
        }
        cout << "Deleted element: " << q[front] << endl;    if
(front == rear)
            front = rear = -1;
        else
            front = (front + 1) % N;
    }
    void display() {
        if (front == -1) {
            cout << "Circular Queue is empty" << endl;    return;
        }
        cout << "Final Circular Queue: ";
        int i = front;
        while (true) {
            cout << q[i] << " ";
            if (i == rear)
                break;
            i = (i + 1) % N;
        }
    }

```

```

    cout << endl;
}
};

int main() {
    CircularQueue q;
    q.enqueue(10);
    q.enqueue(20);
    q.enqueue(30);
    q.enqueue(40);
    q.enqueue(50);
    q.dequeue();
    q.dequeue();
    q.enqueue(60);
    q.enqueue(70);
    q.display();
    return 0;
}

```

Output:

```

PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> g++ lab.cpp -o lab
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> ./lab.exe
Deleted element: 10
Deleted element: 20
Final Circular Queue: 30 40 50 60 70
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> █

```

Experiment-3

(a) Given an integer N, compute the factorial of N using recursion without using any iterative constructs. .

Source Code:

```

#include <iostream>
using namespace std;
int factorial(int n) {
    if (n == 0 || n == 1)
        return 1;
    return n * factorial(n - 1);
}
int main() {
    int N;
    cout << "Enter N: ";

```

```

    cin >> N;
    if (N < 0)
        cout << "Factorial is not defined for negative
numbers.";
    else
        cout << "Factorial of " << N << " = " << factorial(N);
    return 0;
}

```

Output:

```

PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> g++ lab.cpp -o lab
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> ./lab.exe
Enter N: 34
Factorial of 34 = 0
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> █

```

(b) Given an integer N, compute the Nth Fibonacci number using recursion without using any iterative constructs. .

Source Code:

```

#include <iostream>
using namespace std;
int fibonacci(int n) {
    if (n == 0)
        return 0;
    if (n == 1)
        return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
int main() {
    int N;
    cout << "Enter N: ";
    cin >> N;
    cout << "The " << N << "th Fibonacci number = "
<< fibonacci(N);
    return 0;
}

```

Output:

```
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> g++ lab.cpp -o lab
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> ./lab.exe
Enter N: 10
The 10th Fibonacci number = 55
PS C:\Users\vic\OneDrive\Documents\Desktop\LAB 2ND YEAR\Projects> █
```